C FILE HANDLING PRACTICE QUESTIONS (MEDIUM TO ADVANCED)

MEDIUM LEVEL

1. Open a file in read mode and count number of lines, words, and characters.
2. Copy contents of one file to another file.
3. Merge two files into a third file.
4. Reverse the contents of a file and store in another file.
5. Count frequency of a given character in a file.
6. Search a word in a file and print line numbers where it occurs.
7. Remove all blank lines from a file.
8. Convert lowercase letters to uppercase in a file.
9. Print last N lines of a file.
10. Replace a word in a file with another word.
11. Count number of digits, alphabets, and special characters in a file.
12. Read a file and display only unique words.
13. Sort lines of a file alphabetically.
14. Append one file's contents at the end of another file.
15. Find the longest line in a file.

ADVANCED LEVEL

1. Implement your own version of cp command using C file APIs.
2. Implement your own version of wc command.
3. Implement your own version of grep.
4. Implement file encryption and decryption using XOR.
5. Read a binary file and modify specific records.
6. Store student records in a binary file and perform CRUD operations.
7. Randomly access and update the Nth record in a binary file.
8. Implement file locking using fcntl.
9. Detect file type using magic numbers.
10. Implement log file rotation.
11. Create a program to compare two files.
12. Implement tail command using file seek.
13. Read very large files efficiently using buffering.
14. Implement checksum generation for a file.
15. Recover deleted records conceptually.
16. Create an index file for fast searching.
17. Implement persistent key-value storage using files.
18. Simulate a simple file system using files.
19. Implement mmap-based file reading.
20. Track file changes using inode and stat.

EXPERT / INTERVIEW FOCUSED

1. Difference between text and binary files.
2. How stdio buffering works internally.
3. File descriptor vs FILE pointer.
4. Internal working of fopen, fread, fwrite, fclose.
5. Internal working of lseek.
6. Atomicity in file operations.
7. Race conditions in file access.
8. Sparse files and use cases.
9. Hard links and soft links via programs.
10. Mixing read and fread issues.